Polytech Montpellier
Université de Montpellier – Année universitaire 2026–2027

Registre des décisions d'architecture

(ADR)

Application de gestion de festival



Équipe projet (groupe 11)

Kim Etienne – Ines Mazgar – Mathieu Marty

Document associé :	Document d'Architecture Technique (DAT), `dat-v1.3`
Décisions couvertes :	ADR-001 à ADR-013
Date :	24/10/2026
Dépôt Git :	`https://gitlab.polytech.umontpellier.fr/mathieu.marty/awi_kim_mathieu_ines`
Statut :	document vivant, complété à chaque nouvelle décision

Table des matières

1 Introduction **2**
1.1 Objectif du document . . . 2
1.2 Format et règles . . . 2

2 Registre **2**

3 Décisions **3**
3.1 ADR-001 - Rôles hiérarchiques à niveaux et rôle trésorier parallèle . . . 3
3.2 ADR-002 - Validation de la réservation et masquage des informations commerciales en liste blanche . . . 3
3.3 ADR-003 - Facturation réservée au trésorier, facture immuable . . . 4
3.4 ADR-004 - Numéro retiré . . . 5
3.5 ADR-005 - Back-end en quatre couches : routes, contrôleurs, services, dépôts SQL 5
3.6 ADR-006 - Authentification par jeton JWT dans un cookie HttpOnly . . . 6
3.7 ADR-007 - Facture : instantané des lignes et immuabilité garantie en base . . . 6
3.8 ADR-008 - Contrôle de capacité par verrouillage transactionnel des zones . . . 7
3.9 ADR-009 - Statut « considéré absent » calculé à la lecture . . . 7
3.10 ADR-010 - Déploiement par images versionnées, Docker rootless sur la VM . . . 8
3.11 ADR-011 - Table unique `structure` à indicateurs de rôle ; la réservation porte l'exposant . . . 9
3.12 ADR-012 - Stockage des PDF : règles en fichiers, factures regénérées . . . 9
3.13 ADR-013 - Niveau hiérarchique stocké dans le compte : `utilisateur.niveau` et `est_tresorier` . . . 10

4 Points restant à confirmer **11**

1 Introduction

1.1 Objectif du document

Ce registre rassemble les *Architecture Decision Records* (ADR) du projet. Un ADR consigne une décision d'architecture coûteuse à défaire : le contexte qui l'a rendue nécessaire, les critères fixés avant la comparaison, les options réellement envisagées, la décision retenue et ses conséquences, positives et négatives. Là où le DAT explique *comment* le système fonctionne, les ADR expliquent *pourquoi* il est ainsi.

Ce registre complète le DAT (`dat-v1.3`), dont il reprend les fiches en annexe A ; les deux documents sont produits à partir du même fichier `adr-fiches.tex`, pour ne jamais diverger. Les ADR sont aussi versionnés un par un dans le dépôt, dans `docs/adr/`.

1.2 Format et règles

Identifiant	`ADR-NNN`, dans l'ordre chronologique ; un numéro n'est jamais réutilisé.
Statut	*Proposée* (à valider par l'équipe), *Acceptée*, *Rejetée*, *Remplacée par ADR-n*.
Contexte	le problème à résoudre et les contraintes, de façon factuelle.
Critères	ce qui compte pour ce choix, fixé *avant* de comparer les options.
Options	au moins deux options réelles, évaluées au regard des critères.
Décision	l'option retenue et la raison principale.
Conséquences	positives *et* négatives.

Règles suivies :

- un ADR accepté n'est jamais réécrit : un changement d'avis donne un nouvel ADR qui le remplace, et une information nouvelle qui confirme la décision est ajoutée sous forme de *note datée* ;
- une correction d'erreur (diagramme, faute) n'est pas un ADR : elle est tracée dans le journal des évolutions du DAT ;
- les choix imposés par l'énoncé (Angular, Node.js et TypeScript, PostgreSQL sans ORM, Docker Compose, GitLab CI/CD, VM fournie) sont des contraintes et ne donnent pas lieu à un ADR ;
- les hypothèses à valider par le client sont signalées par la mention *[à confirmer : ...]*.

2 Registre

Tableau 1 – Registre des décisions d'architecture

ADR	Décision	Statut	Date
ADR-001	Rôles hiérarchiques à niveaux et rôle trésorier parallèle	Acceptée	14/10/2026
ADR-002	Validation de la réservation et masquage en liste blanche	Acceptée	14/10/2026
ADR-003	Facturation réservée au trésorier, facture immuable (note du 23/10)	Acceptée	14/10/2026
ADR-004	Numéro retiré : correction d'erreur tracée dans le journal	Retiré	23/10/2026
ADR-005	Back-end en quatre couches (routes, contrôleurs, services, dépôts SQL)	Acceptée	24/10/2026
ADR-006	Authentification par JWT dans un cookie HttpOnly	Proposée	23/10/2026
ADR-007	Facture : instantané des lignes, immuabilité garantie en base	Proposée	23/10/2026
ADR-008	Contrôle de capacité par verrouillage transactionnel	Proposée	23/10/2026
ADR-009	Statut « considéré absent » calculé à la lecture	Acceptée	24/10/2026
ADR-010	Déploiement par images versionnées, Docker rootless sur la VM	Acceptée	24/10/2026
ADR-011	Table unique `structure` ; la réservation porte l'exposant	Acceptée	24/10/2026
ADR-012	Stockage des PDF : règles en fichiers, factures regénérées	Proposée	24/10/2026
ADR-013	Niveau hiérarchique stocké dans le compte (`niveau`, `est_tresorier`)	Acceptée	24/10/2026

Les décisions les plus coûteuses à défaire sont l'ADR-005 (organisation de tout le code), l'ADR-007 (schéma de facturation, critique pour l'intégrité) et l'ADR-011 (modèle des structures, sur lequel reposent réservations, jeux et suivi). Les ADR *Proposées* doivent être relues et acceptées par l'équipe avant le développement du code concerné.

3 Décisions

3.1 ADR-001 - Rôles hiérarchiques à niveaux et rôle trésorier parallèle

Statut : Acceptée **Date :** 14/10/2026

Contexte

Le brouillon prévoyait six rôles cumulables et indépendants (admin, organisateur, membre de l'organisation, gérant de table, trésorier, visiteur). Le client décrit des rôles « descendants » : chaque rôle inclut les droits des rôles inférieurs, les noms sont libres, plusieurs comptes peuvent partager un rôle, et le trésorier est à part. Le mot « organisateur » désignait à la fois la structure qui réserve et un rôle utilisateur.

Critères de décision (fixés avant la comparaison)

(1) Fidélité au modèle d'inclusion décrit par le client ; (2) vérification simple et uniforme dans l'API ; (3) possibilité de tester automatiquement toute la matrice rôle × route ; (4) coût d'évolution si le client ajoute un rôle.

Options considérées

A. Rôles indépendants cumulables, chaque route listant les rôles autorisés : souple, mais l'inclusion doit être répétée sur chaque route, et un oubli crée une faille ou un blocage (critères 1 et 2 mal couverts).

B. Niveaux entiers ordonnés dans le code (`admin`=5 … `membre_orga`=1) et rôle `tresorier` hors hiérarchie : une seule comparaison par route, inclusion garantie par construction ; un droit « en diagonale » n'est pas exprimable.

Décision

Option B. Niveau effectif d'un compte = maximum de ses rôles hiérarchiques ; le trésorier s'ajoute sans changer ce niveau. Le rôle utilisateur « organisateur » devient « commercial » ; « organisateur » n'est plus utilisé (la structure qui réserve est l'*exposant*, ADR-011). Le dernier administrateur ne peut être ni supprimé ni rétrogradé. Le visiteur consulte sans compte (confirmé par le client, Q26).

Conséquences

Positives : intergiciels `exigerNiveau(n)` et `exigerRole('tresorier')` ; matrice testable en CI ; vocabulaire clarifié. *Négatives* : ajouter un rôle intermédiaire oblige à renuméroter les niveaux et à relire toutes les routes ; un droit transverse (par exemple un membre qui saisirait des jeux sans voir le plan) demandera un nouvel ADR ; le niveau étant embarqué dans le jeton (ADR-006), un changement de rôle ne prend effet qu'à la reconnexion. Le rôle « gérant de table » disparaît, donc `reservation_jeu.gerant_id` est supprimé.

Note du 24/10/2026 : le stockage du niveau est précisé par l'ADR-013 (une colonne `niveau` et un indicateur `est_tresorier` dans `utilisateur`) ; le principe de cette décision est inchangé.

3.2 ADR-002 - Validation de la réservation et masquage des informations commerciales en liste blanche

Statut : Acceptée **Date :** 14/10/2026

Contexte

À partir du gestionnaire de plan, les rôles ne doivent voir que la réservation finale : ni remises, ni prix, ni notes, ni statut du workflow, ni contacts de suivi. Les jeux sont ajoutés bien après la négociation, une fois les tables arrêtées.

Critères de décision (fixés avant la comparaison)

(1) Aucune fuite possible, y compris en appelant l'API directement ; (2) la règle est appliquée à un seul endroit ; (3) testabilité ; (4) un champ ajouté plus tard ne doit pas fuiter par défaut.

Options considérées

A. Masquage dans le front : les données transitent quand même sur le réseau (critère 1 violé). Rejetée d'emblée.

B. Vues SQL distinctes (`v_reservation_publique`) : garantie au niveau SQL, mais la règle de droits se répartit entre SQL et code, et chaque requête de liste existe en double.

C. Liste noire : retirer les champs interdits dans le service. On part de la réservation complète et on efface ce qui est commercial (`delete r.remise`, `delete r.notes`…). Simple, mais si l'on ajoute plus tard un champ commercial et qu'on oublie de l'effacer, il part vers tous les rôles (critère 4 violé).

D. Liste blanche : ne recopier que les champs autorisés dans le service. Une *projection*, c'est le choix des champs que l'on renvoie. Pour les rôles qui n'ont pas droit aux informations commerciales, le service fabrique une nouvelle réponse en recopiant un à un les seuls champs permis, par exemple `{ id, exposant, tablesParZone, jeux, zonesDuPlan }` ; remises, prix, notes, statut et contacts de suivi n'y figurent tout simplement pas.

Décision

Option D : on écrit la liste de ce qui est *permis* plutôt que celle de ce qui est interdit. Ainsi, un champ ajouté plus tard est caché par défaut : en cas d'oubli, il manque à l'écran (gênant, mais sans danger) au lieu d'être divulgué. Une colonne `validee_le` est ajoutée à la réservation, renseignée par `PATCH /reservations/:id/validation` (commercial). Avant validation, une réservation est invisible pour les niveaux inférieurs à 4 (réponse 404 [1], pour ne pas révéler son existence). L'ajout de jeux n'est possible qu'après validation.

Conséquences

Positives : un seul point de contrôle, nouveau champ masqué par défaut, test de non-fuite en CI. *Négatives* : deux types TypeScript (`ReservationComplete`, `ReservationPublique`) à maintenir ; la modification d'une réservation déjà validée n'est pas encore définie *[à confirmer : faut-il la dé-valider ? que deviennent les jeux placés ?]*.

3.3 ADR-003 - Facturation réservée au trésorier, facture immuable

Statut : Acceptée **Date :** 14/10/2026

Contexte

Le brouillon faisait émettre les factures par l'organisateur et valider par le trésorier. Le client précise que le trésorier crée la facture à partir de la réservation, qu'elle « ne bouge plus », qu'il peut l'annuler, la rembourser et noter son paiement, et qu'un ajout de tables ou de repas produit une nouvelle facture.

Critères de décision (fixés avant la comparaison)

(1) Intégrité comptable : un document envoyé ne change jamais ; (2) traçabilité datée de chaque action ; (3) séparation entre négociation (commercial) et facturation (trésorier) ; (4) simplicité pour le trésorier.

Options considérées

A. Facture recalculée à la volée depuis la réservation : simple, mais une modification de la réservation modifie rétroactivement un document déjà envoyé (critère 1 violé).

B. Facture modifiable tant qu'elle n'est pas payée : moins de factures, mais aucun historique fiable et contraire à la demande du client.

C. Facture immuable, créée uniquement par le trésorier ; toute correction passe par une annulation suivie d'une nouvelle facture.

Décision

Option C. Statuts `emise`, `annulee`, `a_rembourser`, `remboursee`, chacun daté ; « payée

1. Codes HTTP : **404** : ressource introuvable.

» est dérivé des paiements (ADR-007). Endpoints `POST /reservations/:id/factures`, `PATCH /factures/:id/annulation`, `PATCH /factures/:id/remboursement`. La façon dont le commercial signale un ajout au trésorier reste à préciser.

Conséquences

Positives : historique complet, droits simples à tester. *Négatives* : plusieurs factures par réservation, donc un reste à payer calculé sur l'ensemble ; le trésorier doit savoir qu'il faut refacturer, d'où un indicateur « réservation modifiée depuis la dernière facture » dans le suivi ; le sort d'un paiement partiel sur une facture annulée reste à définir *[à confirmer : report du paiement sur la nouvelle facture ?]*.

Note du 23/10/2026 (entrevue, Q20) : le signalement se fait en interne, hors application, mais l'application doit dater chaque action. Cela confirme la décision sans la modifier : chaque statut de facture porte sa date.

3.4 ADR-004 - Numéro retiré

Statut : Retiré (ce n'était pas une décision) **Date :** 23/10/2026

Ce qui s'est passé

Le 14/10/2026, l'équipe avait créé une fiche ADR-004 intitulée « Correction du schéma des entités ». Le diagramme de l'époque comportait une erreur : il reliait une réservation à une seule facture, et les paiements directement à la réservation. Or le texte et le SQL prévoyaient déjà plusieurs factures par réservation et des paiements rattachés à une facture. La fiche corrigeait donc le dessin.

Pourquoi ce n'est pas un ADR

Un ADR consigne un *choix* entre plusieurs options, avec des critères et des conséquences. Ici, il n'y avait rien à choisir : le diagramme était faux et il fallait le corriger. Une correction d'erreur se note dans le journal des évolutions du DAT, comme le prévoit la trame, et c'est là qu'elle figure désormais.

Pourquoi garder le numéro

Le numéro ADR-004 a déjà été utilisé, dans le dépôt et dans la version `dat-v1`. Un numéro d'ADR n'est jamais réutilisé, pour que chaque référence désigne toujours la même fiche : on garde donc cette fiche « retirée », et la décision suivante porte le numéro ADR-005.

3.5 ADR-005 - Back-end en quatre couches : routes, contrôleurs, services, dépôts SQL

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

Contexte

Express n'impose aucune organisation. Le SQL est écrit à la main (pas d'ORM). Les règles métier sont denses (prix, remises, capacité, masquage, workflow). L'équipe compte trois personnes qui codent toutes dans toutes les couches et doivent pouvoir expliquer n'importe quel fichier en soutenance.

Critères de décision (fixés avant la comparaison)

(1) Tester les règles métier sans base ni HTTP ; (2) concentrer le SQL pour relire facilement le paramétrage des requêtes ; (3) une seule responsabilité par couche, facile à expliquer à un nouvel arrivant ; (4) coût de mise en place.

Options considérées

A. Routes Express contenant SQL et logique : démarrage rapide, mais règles dupliquées entre routes, SQL dispersé, tests uniquement de bout en bout (critères 1, 2 et 3 violés).

B. Trois couches : routes, services, dépôts ; la route déclare l'URL *et* traite la requête (validation, appel du service, code HTTP). Moins de fichiers, mais la route mélange deux responsabilités, et le traitement de la requête ne se teste pas sans Express (critère 3 moins bien couvert).

C. Quatre couches : *routes* (associent une URL et une méthode HTTP aux intergiciels de droits et au contrôleur), *contrôleurs* (lisent la requête, la valident avec `zod`, appellent le service, choisissent le code HTTP), *services* (règles métier, sans rien connaître de HTTP), *dépôts* (seuls à contenir du SQL). C'est aussi l'organisation proposée en exemple par la trame du DAT.

Décision

Option C, pour qu'à chaque couche corresponde une seule responsabilité (critère 3), pour un coût faible : un fichier de plus par ressource par rapport à l'option B. Chaque couche n'appelle que la couche inférieure. Une fonction `avecTransaction(async (client) => ...)` ouvre la transaction dans le service et passe le même client `pg` à tous les dépôts concernés.

Conséquences

Positives : fichiers de routes lisibles d'un coup d'œil (la liste des URL de l'API) ; contrôleurs testables sans démarrer Express ; services testables avec de faux dépôts ; SQL relu en un seul endroit (`src/depots`). *Négatives* : cinq fichiers par ressource (route, contrôleur, service, dépôt, types) ; pour les ressources simples, contrôleur et service ne font souvent que relayer, ce que l'on accepte ; les dépôts doivent accepter un client `pg` en paramètre pour participer à une transaction.

3.6 ADR-006 - Authentification par jeton JWT dans un cookie HttpOnly

Statut : Proposée **Date :** 23/10/2026

Contexte

Connexion par login et mot de passe. L'API doit rester sans état pour pouvoir être dupliquée (environ 1 000 lecteurs). Le front et l'API sont servis sous la même origine grâce au reverse proxy. Les menaces principales sont le vol de jeton par XSS et la falsification de requête (CSRF).

Critères de décision (fixés avant la comparaison)

(1) Résistance au XSS ; (2) résistance au CSRF ; (3) API sans état ; (4) simplicité côté Angular ; (5) possibilité de révoquer un accès.

Options considérées

A. Session serveur (cookie opaque, sessions dans PostgreSQL) : révocation immédiate (5), mais état côté serveur et lecture en base à chaque requête (3).

B. JWT dans `localStorage`, envoyé dans l'en-tête `Authorization` : simple, insensible au CSRF, mais lisible par tout script injecté (1 violé).

C. JWT signé (HS256) dans un cookie `HttpOnly; Secure; SameSite=Strict` : illisible par le JavaScript, CSRF neutralisé par `SameSite=Strict` et la même origine, API sans état.

Décision

Option C. Durée de vie 8 h, sans jeton de rafraîchissement ; à l'expiration, l'utilisateur se reconnecte. Le jeton contient l'identifiant, le niveau effectif et l'indicateur trésorier. Mots de passe hachés avec bcrypt. Limitation de débit sur `/auth/login`.

Conséquences

Positives : aucun jeton accessible au JavaScript ; aucune lecture en base pour authentifier. *Négatives* : pas de révocation avant expiration : un compte supprimé ou rétrogradé reste actif jusqu'à 8 h (atténuation possible : vérifier `utilisateur.actif` à chaque requête, au prix d'une lecture indexée, à trancher) ; HTTPS obligatoire en production pour l'attribut `Secure` ; les tests d'API doivent gérer les cookies.

3.7 ADR-007 - Facture : instantané des lignes et immuabilité garantie en base

Statut : Proposée **Date :** 23/10/2026

Contexte

L'ADR-003 impose l'immuabilité, mais le schéma ne stockait que `montant_total` : le PDF aurait été régénéré à partir de la réservation, qui, elle, continue d'évoluer. Le PDF d'une ancienne facture aurait donc changé.

Critères de décision (fixés avant la comparaison)

(1) Reproduire à l'identique une facture déjà envoyée ; (2) immuabilité garantie même en cas de bogue applicatif ; (3) données interrogeables (liste des factures, reste à payer) ; (4) simplicité de sauvegarde.

Options considérées

A. Montant seul et PDF régénéré depuis la réservation : critère 1 violé.

B. Stocker le fichier PDF (volume ou `bytea`) : reproductible, mais binaires à sauvegarder à part et contenu non interrogeable (3, 4).

C. Stocker l'instantané : lignes figées (`facture_ligne` : libellé, quantité, prix unitaire, montant) et coordonnées de l'exposant figées ; PDF généré à la demande depuis cet instantané ; déclencheur PostgreSQL qui refuse toute suppression et toute modification autre que le statut et ses dates.

Décision

Option C. La création de la facture et de ses lignes se fait dans une transaction. Le statut « payée » n'est pas stocké : il est dérivé (somme des paiements encaissés $\geq$ montant), ce qui évite deux sources de vérité.

Conséquences

Positives : un PDF envoyé est toujours reproductible ; un bogue applicatif ne peut pas réécrire une facture. *Négatives* : duplication volontaire de données ; logique en base (déclencheur) à documenter et à tester ; une évolution du gabarit PDF change l'aspect, mais pas le contenu, des anciennes factures.

3.8 ADR-008 - Contrôle de capacité par verrouillage transactionnel des zones

Statut : Proposée **Date :** 23/10/2026

Contexte

Plusieurs commerciaux travaillent en même temps (Q24). Une réservation est refusée si elle dépasse la capacité de la zone ou l'espace restant du festival. Une vérification « lire puis écrire » laisse passer deux réservations simultanées qui lisent chacune deux tables libres et en prennent deux.

Critères de décision (fixés avant la comparaison)

(1) Aucune surréservation, même en concurrence ; (2) message d'erreur clair ; (3) simplicité d'explication ; (4) performance (les écritures sont rares).

Options considérées

A. Vérification applicative sans verrou : condition de course (1 violé).

B. Isolation `SERIALIZABLE` : correcte, mais erreurs de sérialisation à rejouer et comportement difficile à expliquer (3).

C. `SELECT ... FOR UPDATE` sur la ligne du festival puis sur les zones concernées, dans la transaction, avant recalcul et écriture.

Décision

Option C, dans `avecTransaction` ; verrous pris dans un ordre fixe (festival, puis zones par identifiant croissant) pour éviter les interblocages ; réponse `409 Conflict` [2] avec le nombre de tables restantes en cas de dépassement.

Conséquences

Positives : surréservation impossible, logique lisible. *Négatives* : les écritures sur un même festival sont sérialisées (négligeable au volume attendu) ; la règle vit dans le service et non dans le schéma, d'où un test d'intégration concurrent obligatoire dans la CI.

3.9 ADR-009 - Statut « considéré absent » calculé à la lecture

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

2. Codes HTTP : **409** : conflit avec l'état des données (capacité dépassée, facture déjà émise…).

Contexte

Le client veut que les exposants sans réponse passent automatiquement à « considéré absent » à la date du festival (Q11). L'API pourra être dupliquée (section 2).

Critères de décision (fixés avant la comparaison)

(1) Aucun passage manqué (API arrêtée à minuit, redémarrage) ; (2) compatible avec plusieurs instances de l'API ; (3) simplicité ; (4) filtres et tris cohérents.

Options considérées

A. Tâche planifiée dans l'API (`node-cron`) : simple, mais exécutée par chaque instance (2) et manquée si l'API est arrêtée (1).

B. Extension `pg_cron` : fiable, mais extension à ajouter à l'image PostgreSQL et logique cachée en base.

C. Calcul à la lecture dans une vue : `CASE WHEN f.date_debut <= CURRENT_DATE AND r.statut IN ('pas_contacte','contacte','discussion') THEN 'considere_absent' ELSE r.statut END`.

Décision

Option C, vue `v_reservation_statut` utilisée par toutes les lectures. Le statut reste saisissable manuellement avant la date (les notes d'entrevue décrivent un passage manuel « pour ne plus l'avoir dans la liste ») *[à confirmer : statuts concernés par le passage automatique]*.

Conséquences

Positives : toujours exact, rien à planifier. *Négatives* : le statut stocké diffère du statut affiché, toute lecture doit passer par la vue ; la date du passage n'est pas enregistrée (c'est la date du festival).

3.10 ADR-010 - Déploiement par images versionnées, Docker rootless sur la VM

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10, révisée le 24/10/2026)

Contexte

Une VM fournie (`162.38.111.31`), joignable en SSH depuis le réseau de Polytech ou par VPN, avec un compte par membre, sans droits administrateur, et Docker en mode *rootless*. Un pipeline GitLab est imposé et la procédure de retour arrière est évaluée (critère E).

Critères de décision (fixés avant la comparaison)

(1) Retour arrière rapide et sûr ; (2) l'image déployée est exactement celle qui a été testée ; (3) charge limitée sur la VM ; (4) aucun secret dans le dépôt.

Options considérées

A. `git pull` puis `docker compose build` sur la VM : simple, mais compilation sur la VM (3), image non testée (2), retour arrière par recompilation (1).

B. Images construites en CI, poussées dans le registre GitLab avec le SHA court du commit comme étiquette ; la VM fait `pull` puis `up -d` avec `TAG=<sha>`.

C. Fichiers déployés sans Docker en production (`rsync` et gestionnaire de processus) : écart entre développement et production.

Décision

Option B. L'application tourne sous le Docker rootless du responsable du déploiement (`mathieu.marty`) ; la CI s'y connecte avec une clé SSH dédiée, stockée dans une variable GitLab protégée et masquée. Retour arrière : relancer le job `deploy` avec l'étiquette précédente. Migrations SQL appliquées par l'API au démarrage, uniquement additives.

Conséquences

Positives : retour arrière en moins d'une minute, traçabilité commit → image → version en ligne. *Négatives* : dépendance au registre GitLab (quota de stockage) ; un retour arrière applicatif ne défait pas une migration, d'où la règle des migrations additives (on ajoute d'abord, on supprime dans une version ultérieure) ; en Docker rootless, les ports inférieurs à 1024 ne sont pas publiables

sans réglage de l'administrateur *[à confirmer : 80/443 ou ports hauts]*, le démon doit être maintenu après déconnexion (`loginctl enable-linger`), et seul le compte qui l'héberge peut gérer les conteneurs ; la VM doit disposer d'un jeton de lecture du registre.

3.11 ADR-011 - Table unique `structure` à indicateurs de rôle ; la réservation porte l'exposant

Statut : Acceptée **Date :** 24/10/2026 (proposée le 23/10/2026)

Contexte

Les données fournies décrivent des éditeurs avec deux indicateurs, `exposant` et `distributeur` (164 exposants, 3 cumulent les deux, 143 ni l'un ni l'autre). Le client introduit la notion d'*exposant* : l'association se réserve des espaces (ni jeux ni facture) ; un prestataire, qui n'est pas éditeur, loue un espace pour animer les jeux d'un ou plusieurs éditeurs (facturé, avec jeux). Le brouillon appelait « organisateur » la structure qui réserve et « animateur » une structure qui « distribue ».

Critères de décision (fixés avant la comparaison)

(1) Pas de doublon d'une même entreprise ni de ses contacts ; (2) couvrir tous les cas d'exposant du client ; (3) requêtes de suivi (UC6) simples ; (4) contraintes vérifiables.

Options considérées

A. Tables séparées `editeur`, `exposant`, `distributeur` : explicite, mais doublons d'entreprises et de contacts (1).

B. Héritage (`structure` et tables filles) : propre en théorie, mais jointures systématiques pour très peu d'attributs propres à chaque rôle.

C. Table unique `structure` avec `est_editeur`, `est_distributeur`, `est_exposant`, `est_association` ; `reservation.exposant_id` ; `reservation.facturable` ; les jeux d'une réservation peuvent appartenir à n'importe quel éditeur.

Décision

Option C, sous réserve de validation par le client. « Animateur » est renommé « distributeur », conformément aux données *[à confirmer : sens exact de « distributeur » et de `jeu.distributeur_id`]*. Une seule structure peut porter `est_association` (index unique partiel).

Conséquences

Positives : une entreprise, une fiche ; les trois cas d'exposant tiennent dans le même modèle. *Négatives* : la cohérence entre jeux présentés et exposant (un prestataire anime-t-il seulement pour des éditeurs déclarés ?) n'est pas contrainte ; colonnes renommées par rapport au brouillon (`organisateur_id` → `exposant_id`).

3.12 ADR-012 - Stockage des PDF : règles en fichiers, factures regénérées

Statut : Proposée **Date :** 24/10/2026

Contexte

L'application manipule deux sortes de PDF. Les **règles des jeux** sont des fichiers déposés tels quels par le gestionnaire de jeux : l'application ne les lit pas, elle les garde et les renvoie. Les **factures** sont produites par l'application à partir de ses propres données.

Critères de décision (fixés avant la comparaison)

(1) Base de données légère, donc requêtes et sauvegardes rapides ; (2) une facture déjà envoyée doit pouvoir être reproduite à l'identique ; (3) simplicité ; (4) peu de données à sauvegarder à part.

Options considérées

A. Tout mettre dans la base (colonne binaire `bytea`) : une seule sauvegarde, mais la base grossit de plusieurs gigaoctets avec environ 1 500 jeux (critère 1).

B. Tout mettre en fichiers, factures comprises : simple à servir, mais les factures deviennent des fichiers à sauvegarder à part et ne sont plus interrogeables (critères 3 et 4).

C. Règles en fichiers, factures regénérées : les règles vont dans le volume `regles` et la base ne garde que leur nom (`jeu.regles_pdf`) ; une facture n'est pas stockée en PDF, elle est regénérée à chaque téléchargement depuis ses données figées en base (ADR-007).

Décision

Option C. Chaque sorte de PDF est rangée selon sa nature : un fichier reçu reste un fichier, un document que l'application fabrique est refait à partir des données.

Conséquences

Positives : base légère ; factures toujours reproductibles et interrogeables (liste, reste à payer). *Négatives* : le volume `regles` doit être sauvegardé en plus de la base ; l'API doit supprimer le fichier quand une règle est remplacée ou supprimée ; si la mise en page des factures change, les anciennes factures changent d'aspect (pas de contenu) *[à confirmer : faut-il conserver le fichier exact envoyé à l'exposant ?]*.

3.13 ADR-013 - Niveau hiérarchique stocké dans le compte : `utilisateur.niveau` et `est_tresorier`

Statut : Acceptée **Date :** 24/10/2026

Contexte

L'ADR-001 fixe des rôles hiérarchiques à niveaux et un rôle trésorier parallèle. Le premier script SQL stockait les rôles d'un compte dans une table `utilisateur_role`, une ligne par rôle, et calculait le niveau comme le plus élevé de ces rôles. Or, avec des rôles hiérarchiques, posséder deux rôles hiérarchiques n'apporte rien : le plus élevé inclut l'autre. De plus, un compte qui n'avait que le rôle trésorier avait un niveau nul, et les routes de lecture le bloquaient, alors que le client veut qu'il voie tout comme un membre (Q21).

Critères de décision (fixés avant la comparaison)

(1) Fidélité à l'ADR-001 ; (2) rendre impossibles en base les états absurdes (compte sans niveau, trésorier sans accès en lecture) ; (3) simplicité des requêtes et du contrôle d'accès ; (4) facilité d'explication.

Options considérées

A. Table `utilisateur_role` (une ligne par rôle, niveau calculé) : souple, mais permet des combinaisons inutiles et le trésorier sans niveau (critère 2), et demande une jointure à chaque connexion (critère 3).

B. Colonne `niveau` (1 à 5) et indicateur `est_tresorier` dans `utilisateur` : un seul niveau par compte, obligatoire ; le trésorier s'ajoute par un booléen.

C. Table de référence des rôles (nom, niveau) et clé étrangère : noms des rôles en base, mais une table de plus pour six valeurs stables, déjà écartée par l'ADR-001 (option RBAC).

Décision

Option B. `niveau` est `NOT NULL` avec `CHECK (niveau BETWEEN 1 AND 5)` ; les noms des rôles ne figurent que dans le code. Un trésorier a donc toujours au moins le niveau 1 (membre), ce qui règle le blocage de la lecture.

Conséquences

Positives : une seule lecture à la connexion ; un compte a toujours un niveau valide ; `exigerNiveau(n)` compare un entier, `exigerRole('tresorier')` lit un booléen. *Négatives* : ajouter un rôle intermédiaire oblige à renuméroter les niveaux existants par une migration ; un futur rôle parallèle autre que trésorier demanderait une nouvelle colonne.

4 Points restant à confirmer

Auprès du client

— **ADR-002** : que se passe-t-il si une réservation validée est modifiée ? Faut-il la dé-valider, et que deviennent les jeux déjà placés ?
— **ADR-003** : un paiement partiel sur une facture annulée est-il reporté sur la nouvelle facture ?
— **ADR-009** : quels statuts passent automatiquement à « considéré absent » ?
— **ADR-012** : faut-il conserver le fichier PDF exact de chaque facture envoyée ?
— **ADR-011** : sens exact de « distributeur » ; un prestataire anime-t-il seulement pour des éditeurs déclarés ?

Sur la VM ou auprès de l'administrateur des VM

— **ADR-010** : les ports 80 et 443 sont-ils publiables en Docker rootless, ou faut-il des ports hauts ?
— **ADR-010** : le *linger* est-il autorisé pour le compte qui héberge l'application ?
— **ADR-010** : le runner GitLab peut-il joindre la VM en SSH ?
— **ADR-006** : quel certificat HTTPS utiliser (certificat de l'école ou auto-signé), puisque Let's Encrypt n'est pas possible ?